DATABASE MANAGEMENT SYSTEM (CT 652)

Tribhuvan University | Institute of Engineering | BCT | Year III / Part II

CHAPTER 7: Object-Based Databases and XML

Syllabus Hours: 6 hrs | Marks Weight: 12 marks

Complete Exam-Ready Answers | All Past Questions Covered (2070-2082)

CHAPTER 7 SYLLABUS TOPICS

- **7.1** Complex Data Types
- **7.2** Structured Types and Inheritance in SQL
- **7.3** Table Inheritance
- **7.4** Array and Multiset Types in SQL
- **7.5** Object Identity and Reference Types in SQL
- **7.6** Implementing O-R Features
- **7.7** Persistent Programming Languages
- **7.8** Object-Relational Mapping (ORM)
- **7.9** XML - Structure and Features
- **7.10** XML Document Schema - DTD and XML Schema
- **7.11** Querying and Transforming XML Data - XPath, XQuery, XSLT
- **7.12** XML Applications and Storage in DBMS

SECTION 1: Object-Relational Model - Complex Data Types

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q1. What are Complex Data Types in the Object-Relational Model? Why does the relational model need to be extended? Explain the limitations of the traditional relational model.

[2+2+4]
2081 Chaitra 2079 Chaitra 2075 Bhadra

Answer: Complex Data Types and the Object-Relational Model

The traditional relational model enforces First Normal Form (1NF) - all attribute values must be ATOMIC (single, indivisible values). While this works well for simple business data, modern applications (multimedia, engineering, GIS, bioinformatics) need to store complex, non-atomic data. The Object-Relational (O-R) model extends the relational model with object-oriented features while retaining SQL as the query language.

Limitations of the Traditional Relational Model:

Limitation	Description	Example of Problem
Only Atomic Values (1NF)	Attributes cannot hold sets, lists, arrays, or nested structures.	Cannot store a list of phone numbers directly in one column.
No User-Defined Types	Only built-in types (INT, VARCHAR, DATE, etc.) are allowed.	Cannot define a "Point" type with x and y coordinates for GIS data.

No Inheritance	Cannot model is-a relationships between types (e.g., Employee is-a Person).	Must duplicate common attributes in every subtype table.
No Methods/Behavior	Attributes store data only - no associated operations or functions.	Cannot attach a "distance()" method to a geographic coordinate type.
No Object Identity	Two rows with identical attribute values are considered duplicate - no notion of distinct objects.	In object model, two Point(1,1) objects can be distinct objects.
Poor Support for Complex Objects	Documents, images, CAD drawings, and molecular structures cannot be modeled naturally.	Storing a molecule (atoms + bonds) requires multiple unnatural joins.

The Object-Relational (O-R) Solution:

The Object-Relational model (standardized in SQL:1999 and extended in SQL:2003, SQL:2011) extends SQL with:

Extension	Description
Complex (Structured) Types	User-defined types (UDTs) with multiple attributes. Can be used as column types in tables.
Collection Types	Array, Multiset (bag), and List types - store multiple values in a single attribute.
Inheritance	Type inheritance (subtypes/supertypes) and table inheritance (sub-tables/super-tables).
Reference Types (REF)	Attributes that hold references (object IDs) pointing to rows in other tables - like foreign keys but richer.
Methods / Functions	User-defined functions and procedures associated with types.
Large Object Types (LOB)	BLOB (Binary Large Object) and CLOB (Character Large Object) for images, videos, long text.

Q2. What are Structured Types (User-Defined Types) in SQL? Explain how to CREATE and USE structured types. Show type inheritance with SQL examples.

[2+4+2]
2081 Ashwin 2079
Chaitra 2076 Baisakh

Answer: Structured Types and Type Inheritance in SQL

Structured Type (User-Defined Type): A structured type is a named, composite type defined by the user in SQL, consisting of multiple attributes. Once defined, it can be used as the data type of columns in tables, just like INT or VARCHAR. Structured types can have associated methods (functions).

Creating and Using Structured Types:

```
-- Define a structured type for Address
CREATE TYPE Address AS (
    street      VARCHAR(100),
    city        VARCHAR(50),
    pincode     CHAR(6)
) NOT FINAL;

-- Define a structured type for Name
CREATE TYPE Name AS (
    firstname   VARCHAR(50),
    lastname    VARCHAR(50)
) NOT FINAL;

-- Use structured types as column types in a table
CREATE TABLE Person (
    pid         INT             PRIMARY KEY,
    pname       Name,           -- structured type as column
    paddress    Address,        -- structured type as column
    dob         DATE
);

-- Insert a row using structured type constructors
INSERT INTO Person VALUES (
    101,
    Name('Ram', 'Sharma'),       -- Name constructor
    Address('Putalisadak', 'Kathmandu', '44600'),
    ''1998-04-15''
);

-- Query using dot notation to access nested attributes
SELECT pid, pname.firstname, paddress.city
FROM Person
WHERE paddress.city = ''Kathmandu'';
```

Type Inheritance (Subtype / Supertype):

SQL supports type inheritance using the UNDER clause. A subtype INHERITS all attributes and methods of its supertype and can add new attributes or override methods. This models the is-a relationship.

```
-- Supertype: Person type
CREATE TYPE PersonType AS (
    pid         INT,
    pname       VARCHAR(100),
    dob         DATE
) NOT FINAL;            -- NOT FINAL means subtypes can be created
```

```
-- Subtype: Employee EXTENDS Person (inherits pid, pname, dob)
CREATE TYPE EmployeeType UNDER PersonType AS (
    emp_id      INT,
    salary      DECIMAL(10,2),
    dept        VARCHAR(50)
) NOT FINAL;

-- Subtype: Student EXTENDS Person
CREATE TYPE StudentType UNDER PersonType AS (
    roll_no     VARCHAR(20),
    program     VARCHAR(50)
) NOT FINAL;

-- Sub-subtype: Teaching Assistant (is both Employee and Student)
CREATE TYPE TAType UNDER EmployeeType AS (
    thesis_title VARCHAR(200)
) FINAL;   -- FINAL means no further subtypes allowed
```

Type Inheritance Hierarchy
PersonType
(pid, pname, dob)
/ \
EmployeeType StudentType
(+ emp_id, salary, dept) (+ roll_no, program)
|
TAType
(+ thesis_title)
EmployeeType inherits: pid, pname, dob (from PersonType)
adds: emp_id, salary, dept
TAType inherits: pid, pname, dob, emp_id, salary, dept (from EmployeeType)
adds: thesis_title

Note: NOT FINAL allows subtypes. FINAL prevents further subtyping. Subtype objects can be used wherever supertype objects are expected (substitutability / polymorphism).

Q3. What is Table Inheritance in SQL? How does it differ from Type Inheritance? Explain with CREATE TABLE ... UNDER examples and how queries work on supertables and subtables.

[2+4+2]

2080 Ashwin 2078
Chaitra 2073 Bhadra

Answer: Table Inheritance

Table Inheritance: Table inheritance allows a table (subtable) to be defined as an extension of another table (supertable) using the UNDER clause. The subtable INHERITS all columns of the supertable and can add new columns. Every row inserted into a subtable is also automatically visible in the supertable.

Type Inheritance vs Table Inheritance:

Feature	Type Inheritance	Table Inheritance
What is inherited	One TYPE extends another type. Attributes and methods are inherited.	One TABLE extends another table. All columns of supertable are inherited.
Applies to	Data types (used as column types).	Entire tables (actual stored relations).
SQL Syntax	CREATE TYPE SubType UNDER SuperType AS (...)	CREATE TABLE SubTable OF SubType UNDER SuperTable
Data Visibility	Not directly about stored rows.	Rows in subtable appear in supertable queries too.
Real-world analogy	Class inheritance in OOP.	Polymorphic table access in RDBMS.

```
-- Supertable: Person (uses PersonType)
CREATE TABLE Person OF PersonType (
    PRIMARY KEY (pid)
);

-- Subtable: Employee UNDER Person (uses EmployeeType which extends
PersonType)
CREATE TABLE Employee OF EmployeeType
UNDER Person;   -- inherits pid, pname, dob columns from Person
                -- adds emp_id, salary, dept columns

-- Subtable: Student UNDER Person
CREATE TABLE Student OF StudentType
UNDER Person;   -- inherits pid, pname, dob; adds roll_no, program

-- Inserting into subtable
INSERT INTO Employee VALUES
    (101, ''Ram Sharma'', ''1995-01-15'', 5001, 50000.00, ''Engineering'');

-- Querying: subtable rows are VISIBLE through supertable
SELECT pid, pname FROM Person;
-- Returns BOTH Employee rows AND Student rows (and any plain Person rows)
```

```
-- Query only employees
SELECT emp_id, salary FROM Employee WHERE dept = ''Engineering'';

-- ONLY query (exclude subtable rows from supertype query)
SELECT pid, pname FROM ONLY(Person);
-- Returns only rows inserted directly into Person, not subtable rows
```

Exam Tip: "ONLY(Person)" returns only direct rows in Person, excluding rows inherited from subtables Employee and Student. Without ONLY, a supertable query returns ALL rows including subtable rows.

SECTION 2: Array, Multiset, Object Identity and Reference Types

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin

Q4. What are Collection Types in SQL? Explain ARRAY and MULTISET types with syntax and examples. How do you query and manipulate collection-valued attributes?

[2+4+2]
2081 Chaitra 2079 Chaitra 2075 Bhadra

Answer: Array and Multiset Types in SQL

Collection Types: Collection types allow a single attribute to hold multiple values of the same type. SQL:1999 introduced ARRAY (ordered, allows duplicates) and MULTISET (unordered, allows duplicates) as collection types. These directly violate 1NF - they are the O-R model's deliberate extension beyond the traditional relational model.

ARRAY Type:

An ARRAY is an ORDERED collection of elements of the same type, accessed by INTEGER INDEX (1-based). The maximum number of elements is specified at definition time.

```
-- Create a table with an ARRAY attribute
CREATE TABLE Employee (
    emp_id      INT PRIMARY KEY,
    ename       VARCHAR(100),
    phone_nos   VARCHAR(15) ARRAY[3],   -- array of up to 3 phone numbers
    skills      VARCHAR(50) ARRAY[10]   -- array of up to 10 skills
);

-- Insert with ARRAY constructor
INSERT INTO Employee VALUES (
    101, ''Ram Sharma'',
    ARRAY[''9841001001'', ''9851001001''],   -- 2 phone numbers
    ARRAY[''Java'', ''Python'', ''SQL'']       -- 3 skills
);
```

```
-- Access element by index (1-based)
SELECT ename, phone_nos[1] AS primary_phone
FROM Employee WHERE emp_id = 101;

-- UNNEST array to treat elements as rows
SELECT emp_id, UNNEST(phone_nos) AS phone
FROM Employee;
-- Returns one row per phone number per employee
```

MULTISET Type:

A MULTISET (also called BAG) is an UNORDERED collection that allows duplicate values. Unlike arrays, elements are not indexed and have no guaranteed order.

```
-- Table with MULTISET attribute
CREATE TABLE Department (
    dept_id     INT PRIMARY KEY,
    dept_name   VARCHAR(100),
    locations   VARCHAR(100) MULTISET   -- multiple locations (no order)
);

-- Insert with MULTISET constructor
INSERT INTO Department VALUES (
    10, 'Computer Science',
    MULTISET[''Kathmandu'', ''Pokhara'', ''Lalitpur'']
);

-- Check if a value is in a multiset
SELECT dept_name FROM Department
WHERE ''Kathmandu'' IN (SELECT value FROM UNNEST(locations) AS T(value));

-- Count elements in a collection
SELECT dept_name, CARDINALITY(locations) AS num_locations
FROM Department;
```

Feature	ARRAY	MULTISET
Order	ORDERED - elements have position (index).	UNORDERED - no guaranteed position.
Access	By integer index: arr[1], arr[2].	No index access. Use UNNEST or IN.
Duplicates	Allowed.	Allowed.
Max Size	Specified at definition: ARRAY[n].	Unbounded (or implementation limit).
Use Case	Phone numbers list, ordered items, matrices.	Unordered tags, location sets, category memberships.

Q5. What is Object Identity in Object-Relational databases? Explain Reference Types (REF) in SQL with examples. How does REF differ from a FOREIGN KEY?

[2+4+2]
2080 Ashwin 2078
Baishakh 2073 Bhadra

Answer: Object Identity and Reference Types

Object Identity (OID): In object-oriented and object-relational databases, every object (row) has a unique SYSTEM-GENERATED identity that is independent of its attribute values. Two objects with identical attribute values are still distinct objects. This is in contrast to the relational model where equality of all attributes means the rows are the same.

Object Identity vs Value Identity:

Feature	Relational Model (Value Identity)	O-R Model (Object Identity)
Uniqueness	Two rows are equal if ALL attribute values are equal.	Every object has a unique OID regardless of attribute values.
Mutation	Changing an attribute value creates a "different" row.	Changing attributes does not change the object's identity (same OID).
References	Foreign keys copy the referenced key value.	REF stores the OID of the referenced object directly.
Dangling References	FK points to non-existent row if parent is deleted (DBMS enforces).	REF can become dangling if referenced object deleted (must handle explicitly).

Reference Types (REF) in SQL:

REF Type: A REF attribute stores an OBJECT IDENTIFIER that references (points to) a specific row (object) in another table. It is a typed pointer - REF(PersonType) can only point to rows of PersonType objects.

```
-- First define the referenced type and table
CREATE TYPE DepartmentType AS (
    dept_id     INT,
    dept_name   VARCHAR(100)
) NOT FINAL;

CREATE TABLE Department OF DepartmentType (
    REF IS dept_oid SYSTEM GENERATED,   -- system generates OID
automatically
    PRIMARY KEY (dept_id)
);
```

```
-- Employee type references Department via REF
CREATE TYPE EmployeeType AS (
    emp_id      INT,
    ename       VARCHAR(100),
    salary      DECIMAL(10,2),
    works_in    REF(DepartmentType) SCOPE Department  -- typed reference
) NOT FINAL;

CREATE TABLE Employee OF EmployeeType (
    PRIMARY KEY (emp_id)
);

-- Insert Employee with a REF to a Department
INSERT INTO Employee
SELECT 101, 'Ram', 50000, dept_oid   -- get OID of CS department
FROM Department WHERE dept_name = 'Computer Science';

-- Query using arrow notation (->) to navigate references (dereference)
SELECT e.ename, e.works_in->dept_name AS department
FROM Employee e;
-- e.works_in is a REF; -> dereferences it to get the Department row
```

REF vs FOREIGN KEY Comparison:

Feature	Foreign Key (FK)	Reference Type (REF)
What is stored	A COPY of the referenced key value (e.g., dept_id = 10).	The OID (internal system pointer) of the referenced object.
Navigation	Must use JOIN to navigate.	Use arrow notation -> to dereference directly (no explicit join).
Scope	Can reference any table with matching type.	Typed + scoped: REF(DepartmentType) SCOPE Department.
Cascading Rules	ON DELETE CASCADE/SET NULL/RESTRICT.	No automatic cascading. Can become dangling references.
Relational compatibility	Core relational feature - all RDBMS support.	Object-Relational extension - SQL:1999 standard.
Performance	Join may require additional I/O.	Direct OID navigation - potentially faster.

SECTION 3: Object-Relational Mapping (ORM) and Persistent Programming Languages

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin

Q6. What is Impedance Mismatch in database programming? What are Persistent Programming Languages? How do they differ from Embedded SQL for database access?

[2+4+2]
2081 Chaitra 2079 Chaitra 2076 Baisakh

Answer: Impedance Mismatch and Persistent Programming Languages

Impedance Mismatch: Impedance mismatch is the fundamental incompatibility between the data models and type systems of programming languages (OOP: objects, classes, inheritance, methods) and relational databases (relations, tuples, SQL). Developers must manually convert between the two representations - a tedious, error-prone process.

Sources of Impedance Mismatch:

Source	Programming Language	Relational Database	Problem
Data Model	Objects with methods and inheritance.	Tables with rows and columns.	Objects must be mapped to rows; methods are not stored in DB.
Type System	Rich types: classes, generics, collections.	Only SQL types: INT, VARCHAR, DATE, etc.	Complex objects must be flattened into rows; collections need separate tables.
Identity	Objects have reference identity (two objects with same values are distinct).	Rows identified only by primary key value.	Object identity must be mapped to PK; references become FKs.
Relationships	Navigated via object references (one.two.three).	Navigated via JOINs.	Every relationship traversal requires a JOIN (costly; verbose code).
Granularity	Fine-grained objects (Address with street, city, zip).	Coarse-grained rows; all attributes at same level.	Nested objects must be flattened into the same row or split across tables.

Persistent Programming Language: A persistent programming language is a programming language extended with the ability to persist objects (store them in long-term storage across program executions) WITHOUT requiring explicit data conversion. The language natively understands database storage - objects automatically persist like database records. Examples: Persistent Java, POET, Versant.

Persistent Programming Languages vs Embedded SQL:

Feature	Embedded SQL (Traditional)	Persistent Programming Languages
Data Model	Mismatch: program uses objects; DB stores tables.	Unified: program objects ARE the database objects. No conversion.
Query Language	Separate SQL language embedded with EXEC SQL prefix.	Host language itself is extended with persistence/query capabilities.
Type System	Two type systems: program types + SQL types.	One unified type system: language types extend to DB storage.
Conversion	Manual conversion between ResultSet and program objects required.	Automatic - no conversion. Objects directly fetched from DB.
Safety	Late binding - SQL errors discovered at RUNTIME.	Type-safe - DB access errors caught at COMPILE TIME.
Complexity	Complex - need SQLCA, cursors, host variables.	Simpler - access DB objects like regular in-memory objects.
Performance	Query execution planned by DB optimizer.	Persistent languages may bypass optimizer - can be less efficient.
Practical Usage	Used in all commercial RDBMS (Oracle, MySQL, PostgreSQL).	Academic/niche - replaced by ORM frameworks in practice.

Q7. What is Object-Relational Mapping (ORM)? Explain the concept with examples. What are the advantages and disadvantages of ORM frameworks? Name popular ORM frameworks.

[2+4+2]
2081 Ashwin 2079 Chaitra 2075 Bhadra

Answer: Object-Relational Mapping (ORM)

Object-Relational Mapping (ORM): ORM is a programming technique that automatically maps between object-oriented program objects (classes) and relational database tables. An ORM framework handles the conversion between objects in memory and rows in the database, eliminating the need for manual SQL writing for CRUD operations. Developers work with objects; the ORM generates SQL behind the scenes.

How ORM Works - Mapping Concepts:

OOP Concept	Maps To	Database Concept
Class (e.g., Employee)	---->	Table (e.g., EMPLOYEE table)
Object (instance of Employee)	---->	Row (one record in EMPLOYEE table)

Attribute (e.g., name, salary)	---->	Column (e.g., NAME, SALARY column)
Object Reference (e.g., e.department)	---->	Foreign Key (e.g., DEPT_ID FK in EMPLOYEE)
Collection (e.g., employee.projects)	---->	Join Table or One-to-Many FK relationship
Class Hierarchy (Employee extends Person)	---->	Table per hierarchy, table per type, or table per class strategies

ORM Example - Java with JPA (Jakarta Persistence API):

```
// ORM Mapping in Java (JPA/Hibernate annotations)

@Entity
@Table(name = "EMPLOYEE")           // maps to EMPLOYEE table
public class Employee {
    @Id
    @Column(name = "EMP_ID")         // maps to EMP_ID column
    private int empId;

    @Column(name = "ENAME")
    private String ename;

    @Column(name = "SALARY")
    private double salary;

    @ManyToOne                       // many employees in one department
    @JoinColumn(name = "DEPT_ID")    // FK column in EMPLOYEE table
    private Department department;   // object reference, not FK int

    // Getters and setters...
}

// Using ORM - no SQL needed for basic operations:
EntityManager em = emf.createEntityManager();

// INSERT: just persist the object
Employee e = new Employee(101, "Ram", 50000.0, csDept);
em.persist(e);   // ORM generates: INSERT INTO EMPLOYEE VALUES (101, 'Ram',
50000, 10)

// SELECT by primary key
Employee e2 = em.find(Employee.class, 101);
// ORM generates: SELECT * FROM EMPLOYEE WHERE EMP_ID = 101
```

```
// Navigating relationships - no JOIN needed
String deptName = e2.getDepartment().getDeptName();
// ORM generates the JOIN automatically when department is accessed (lazy
loading)
```

Advantages and Disadvantages of ORM:

Advantages	Disadvantages
Eliminates repetitive SQL code. Less boilerplate.	ORM-generated SQL may be inefficient. Hard to optimize.
Developer works with familiar OOP paradigm.	Learning curve for ORM framework configuration.
Database-independent. Switch DBMS by changing config.	Complex queries (aggregations, complex joins) still need native SQL or JPQL.
Automatic mapping reduces bugs in data conversion.	"N+1 Query Problem" - ORM may generate too many queries.
Caching, lazy loading, transactions handled automatically.	Overhead of ORM layer can hurt high-performance scenarios.
Schema generation and migration support.	Debugging ORM-generated SQL requires expertise.

Popular ORM Frameworks:

Language	ORM Framework	Notes
Java	Hibernate / JPA (Jakarta Persistence API)	Most widely used Java ORM. JPA is the standard; Hibernate is the most popular implementation.
Python	SQLAlchemy, Django ORM	SQLAlchemy for standalone apps; Django ORM built into Django web framework.
C# / .NET	Entity Framework (EF Core)	Microsoft's official ORM for .NET applications.
Ruby	Active Record (Ruby on Rails)	Built into Rails framework. Convention over configuration philosophy.
PHP	Doctrine, Eloquent (Laravel)	Doctrine is full-featured; Eloquent is simpler and built into Laravel.
Node.js	Sequelize, Prisma, TypeORM	Various options for JavaScript/TypeScript backend development.

SECTION 4: XML - eXtensible Markup Language

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q8. What is XML? Explain the structure and components of an XML document. What are the features of XML? How does XML differ from HTML?

[2+4+2]
2081 Chaitra 2079 Chaitra 2075 Bhadra

Answer: XML - eXtensible Markup Language

XML (eXtensible Markup Language): XML is a text-based, platform-independent markup language developed by the W3C (1998) for storing and transporting structured data. Unlike HTML (which has fixed tags for display), XML uses CUSTOM TAGS defined by the user or application. XML is both human-readable and machine-readable, making it ideal for data exchange between heterogeneous systems.

Structure of an XML Document:

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- XML Declaration: version and encoding -->

<university>
  <!-- ROOT ELEMENT: every XML document has exactly ONE root element -->

  <student id="S101" status="active">
    <!-- ELEMENT with ATTRIBUTES (id, status) -->

    <name>
      <firstname>Ram</firstname>     <!-- CHILD ELEMENTS -->
      <lastname>Sharma</lastname>
    </name>

    <dob>1998-04-15</dob>            <!-- TEXT CONTENT -->

    <address>
      <street>Putalisadak</street>
      <city>Kathmandu</city>
      <pincode>44600</pincode>
    </address>

    <courses>
      <course cid="CS101" credits="3">Database Systems</course>
      <course cid="CS102" credits="4">Operating System</course>
    </courses>
```

```
  </student>

</university>
```

Components of an XML Document:

Component	Description	Example
XML Declaration	Specifies XML version and character encoding. Optional but recommended.	<?xml version="1.0" encoding="UTF-8"?>
Element	The primary building block. Consists of a start tag, content, and end tag.	<city>Kathmandu</city>
Attribute	Name-value pair inside the start tag of an element. Provides metadata about the element.	<student id="S101" status="active">
Text Content	The actual data value between start and end tags.	<firstname>Ram</firstname>
Nested Elements (Children)	An element can contain other elements (hierarchical/tree structure).	<name><firstname>Ram</firstname> </name>
Root Element	The single top-level element that contains all other elements. Exactly ONE root per document.	<university>...</university>
Comment	Human-readable notes; ignored by parser.	<!-- This is a comment -->
CDATA Section	Raw text that should not be parsed as XML markup.	<![CDATA[a < b && c > d]]>
Processing Instruction	Instructions for the XML processor.	<?xml-stylesheet type="text/xsl" href="style.xsl"?>

XML vs HTML Comparison:

Feature	HTML	XML
Purpose	DISPLAY data in a browser (presentation).	STORE and TRANSPORT data (data exchange).
Tags	Predefined fixed tags (<h1>, <p>, <table>, etc.).	User-defined custom tags (you define <student>, <salary>, etc.).
Case Sensitivity	Case-insensitive (<P> = <p>).	Case-sensitive (<Student> != <student>).

Tag Closing	Some tags do not need closing (, <img>).	ALL tags MUST be properly closed. or </br>.
Attributes	Attribute values need not be quoted.	All attribute values MUST be quoted.
Validation	Browsers render invalid HTML (error recovery).	Well-formed XML is strictly required. Parser rejects malformed XML.
Data Focus	Focuses on how data LOOKS.	Focuses on what data MEANS (semantics).

Most Repeated: "What is XML? Explain its structure with an example XML document." is asked almost every year. Always draw a complete XML document with: declaration, root element, child elements, attributes, and text content - using a student or employee example.

Q9. What is a Well-Formed XML document? What is DTD (Document Type Definition)? Write a DTD for a given XML structure. What are the limitations of DTD?

[2+4+2]
2081 Ashwin 2079
Chaitra 2076 Baisakh

Answer: Well-Formed XML, DTD and XML Schema

Well-Formed XML: An XML document is WELL-FORMED if it follows the basic syntactic rules of XML: (1) Has exactly one root element. (2) All tags are properly opened and closed. (3) Tags are properly nested (no crossing). (4) Attribute values are quoted. (5) Case is consistent (XML is case-sensitive). A parser REJECTS any XML that is not well-formed.

Rules for Well-Formed XML:

* Rule 1: Exactly ONE root element. All other elements are children of the root.
* Rule 2: Every start tag must have a corresponding end tag: <name>Ram</name>. Or self-close:
* Rule 3: Elements must be PROPERLY NESTED. <a></a> is correct. <a></a> is WRONG.
* Rule 4: All attribute values must be enclosed in quotes (single or double).
* Rule 5: XML is CASE-SENSITIVE. <Student> and <student> are different elements.
* Rule 6: Special characters must be escaped: < for <, > for >, & for &, " for ".

DTD (Document Type Definition): DTD is a schema definition language for XML. It defines the STRUCTURE (grammar) of an XML document: which elements are allowed, in what order, with what attributes, and with what content. An XML document is called VALID if it is well-formed AND conforms to its associated DTD.

DTD Syntax and Example:

```
<!-- External DTD file: university.dtd -->
```

```
<!-- ELEMENT declarations -->
<!ELEMENT university (student+)>
<!-- university contains ONE OR MORE students: + means 1 or more -->

<!ELEMENT student (name, dob, address, courses?)>
<!-- student contains name, dob, address, and OPTIONAL courses -->
<!-- ?: zero or one    *: zero or more    +: one or more -->

<!ELEMENT name (firstname, lastname)>
<!ELEMENT firstname (#PCDATA)>   <!-- #PCDATA: Parsed Character Data (text)
-->
<!ELEMENT lastname (#PCDATA)>
<!ELEMENT dob (#PCDATA)>

<!ELEMENT address (street, city, pincode)>
<!ELEMENT street (#PCDATA)>
<!ELEMENT city (#PCDATA)>
<!ELEMENT pincode (#PCDATA)>

<!ELEMENT courses (course*)>
<!ELEMENT course (#PCDATA)>

<!-- ATTRIBUTE declarations -->
<!ATTLIST student
    id      ID       #REQUIRED    <!-- id is required, must be unique -->
    status  CDATA    "active"     <!-- status is optional, default "active"
-->
>
<!ATTLIST course
    cid     CDATA    #REQUIRED
    credits CDATA    #IMPLIED     <!-- IMPLIED: optional, no default -->
>
```

DTD Content Models:

Symbol	Meaning	Example
#PCDATA	Parsed Character Data - text content.	<!ELEMENT name (#PCDATA)>
(a, b, c)	Sequence - a then b then c in order.	<!ELEMENT student (name, dob, address)>
(a | b)	Choice - either a OR b.	<!ELEMENT contact (email | phone)>
?	Zero or one occurrence.	<!ELEMENT courses (course?)>
*	Zero or more occurrences.	<!ELEMENT courses (course*)>

+	One or more occurrences.	<!ELEMENT university (student+)>
EMPTY	Element has no content.	<!ELEMENT br EMPTY>
ANY	Element can have any content.	<!ELEMENT note ANY>

Limitations of DTD:

Limitation	Explanation
No data type support	DTD only supports text (#PCDATA) - cannot specify that an element contains an integer, date, or decimal. No type validation.
Limited cardinality expressions	Only ?, *, + for occurrences. Cannot specify exact counts (exactly 3 courses) or ranges (2 to 5).
No namespace support	DTD has no understanding of XML namespaces.
Not XML syntax	DTD uses its own syntax - cannot be processed by XML parsers directly.
No inheritance or reuse	Cannot define base types and extend them. Hard to reuse common definitions.

Exam Tip: DTD vs XML Schema: DTD is simpler but limited. XML Schema (XSD) is more powerful - supports data types, namespaces, complex types, inheritance, and is itself written in XML syntax. Know both for the exam.

Q10. What is XML Schema (XSD)? How does it differ from DTD? Write an XML Schema for a given XML document. Explain the advantages of XML Schema over DTD.

[2+4+2]
2080 Ashwin 2078 Chaitra 2074 Bhadra

Answer: XML Schema (XSD)

XML Schema (XSD - XML Schema Definition): XML Schema is a more powerful alternative to DTD for defining the structure and data types of XML documents. It is itself written in XML syntax, supports rich data types (string, integer, date, decimal, boolean), namespaces, complex type definitions, and type inheritance. Standardized by W3C as a recommendation in 2001.

XML Schema Example (for the same university document):

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <!-- Simple type element: just a string -->
  <xs:element name="firstname" type="xs:string"/>
  <xs:element name="lastname"  type="xs:string"/>
```

```
  <xs:element name="dob"       type="xs:date"/>
  <xs:element name="pincode"   type="xs:string"/>

  <!-- Complex type: Name contains firstname and lastname -->
  <xs:element name="name">
    <xs:complexType>
      <xs:sequence>
        <xs:element ref="firstname"/>
        <xs:element ref="lastname"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

  <!-- Complex type with attributes -->
  <xs:element name="course">
    <xs:complexType>
      <xs:simpleContent>
        <xs:extension base="xs:string">
          <xs:attribute name="cid"     type="xs:string" use="required"/>
          <xs:attribute name="credits" type="xs:integer"/>
        </xs:extension>
      </xs:simpleContent>
    </xs:complexType>
  </xs:element>

  <!-- Student element: minOccurs/maxOccurs for cardinality -->
  <xs:element name="student">
    <xs:complexType>
      <xs:sequence>
        <xs:element ref="name"/>
        <xs:element ref="dob"/>
        <xs:element name="courses">
          <xs:complexType>
            <xs:sequence>
              <xs:element ref="course" minOccurs="0"
maxOccurs="unbounded"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
      <xs:attribute name="id"     type="xs:ID"     use="required"/>
      <xs:attribute name="status" type="xs:string" default="active"/>
    </xs:complexType>
  </xs:element>

  <!-- Root element -->
```

```
  <xs:element name="university">
    <xs:complexType>
      <xs:sequence>
        <xs:element ref="student" minOccurs="1" maxOccurs="unbounded"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

DTD vs XML Schema - Comparison:

Feature	DTD	XML Schema (XSD)
Syntax	Own non-XML syntax.	Written IN XML - can be parsed by any XML processor.
Data Types	Only #PCDATA (text). No type support.	Rich types: xs:string, xs:integer, xs:date, xs:decimal, xs:boolean, etc.
Cardinality	Only ?, *, + symbols.	Precise: minOccurs, maxOccurs="unbounded" or any integer.
Namespace Support	None.	Full namespace support.
Inheritance/Extension	None.	Complex type extension and restriction (inheritance-like).
Reuse / Modularity	Limited.	Named types can be reused across schema. Include and import.
Default Values	Supported for attributes.	Supported for elements AND attributes.
Complexity	Simple and concise.	Verbose but much more expressive.
Practical Use	Legacy systems, simple documents.	Enterprise applications, web services (SOAP/WSDL), most modern XML.

SECTION 5: Querying and Transforming XML - XPath, XQuery, XSLT

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q11. What is XPath? Explain XPath syntax with different types of expressions: absolute paths, relative paths, predicates, axes, and functions. Give examples.

[2+6]
2081 Chaitra 2079 Chaitra 2075 Bhadra

Answer: XPath - XML Path Language

XPath: XPath (XML Path Language) is a W3C standard language for NAVIGATING and SELECTING nodes from an XML document. It uses a path notation (like file system paths) to locate elements, attributes, and text within the XML tree structure. XPath is the foundation for XQuery and XSLT.

XPath Data Model - Node Types:

Node Type	Description	Example
Document (Root) Node	The invisible root node of the entire XML tree. Parent of the root element.	The document itself.
Element Node	Corresponds to an XML element (tag).	<student>, <name>, <city>
Attribute Node	Corresponds to an XML attribute.	id="S101" in <student id="S101">
Text Node	The text content inside an element.	"Ram" in <firstname>Ram</firstname>
Comment Node	XML comment.	<!-- This is a comment -->
Processing Instruction Node	XML processing instruction.	<?xml-stylesheet ...?>

XPath Path Expressions:

```
XML document used for examples:
  <university>
    <student id="S101">
      <name><firstname>Ram</firstname><lastname>Sharma</lastname></name>
      <dob>1998-04-15</dob>
      <address><city>Kathmandu</city><pincode>44600</pincode></address>
    </student>
    <student id="S102">
      <name><firstname>Sita</firstname><lastname>Thapa</lastname></name>
      <dob>1999-08-22</dob>
```

```
      <address><city>Pokhara</city><pincode>33700</pincode></address>
    </student>
  </university>
```

XPath Expression	Result / Meaning
/university	Selects the root element <university>.
/university/student	Selects ALL <student> elements directly under <university>.
//student	Selects ALL <student> elements ANYWHERE in the document (any depth).
//student/name/firstname	Selects all <firstname> elements inside <name> inside <student>.
/university/student[1]	Selects the FIRST <student> element (XPath index is 1-based).
/university/student[last()]	Selects the LAST <student> element.
//student[@id]	Selects all <student> elements that HAVE an id attribute.
//student[@id="S101"]	Selects the <student> element where id attribute equals "S101".
//student[address/city="Kathmandu"]	Selects students whose city is Kathmandu.
//student/name/firstname/text()	Selects the TEXT CONTENT of all <firstname> elements.
//student/@id	Selects the value of the id ATTRIBUTE of all <student> elements.
count(//student)	Returns the COUNT (number) of student elements.
//student[position()<=2]	Selects the first 2 students.
//city[contains(., "a")]	Selects <city> elements whose text content contains the letter "a".

XPath Axes - Navigating the Tree:

Axis	Description	Example
child::	Selects direct children of context node. Default axis.	child::student = student
parent::	Selects the parent of context node.	parent::* = parent element
ancestor::	Selects all ancestors (parent, grandparent...).	ancestor::university
descendant::	Selects all descendants (children, grandchildren...).	descendant::firstname

following-sibling::	Selects all siblings AFTER context node.	following-sibling::student
preceding-sibling::	Selects all siblings BEFORE context node.	preceding-sibling::course
attribute::	Selects attributes. Abbreviated as @.	attribute::id = @id
self::	Selects the context node itself.	self::student

Q12. What is XQuery? How does it extend XPath? Explain the FLWOR expression with examples. Write XQuery to query an XML document.

[2+4+4]
2082 Kartik 2079
Chaitra 2077 Chaitra

Answer: XQuery - XML Query Language

XQuery: XQuery is a powerful, Turing-complete query language for XML data, standardized by W3C. It extends XPath with full query capabilities including iteration (FOR), filtering (WHERE), variable binding (LET), ordering (ORDER BY), and result construction (RETURN). XQuery is to XML what SQL is to relational databases.

FLWOR Expression (The Core of XQuery):

FLWOR (pronounced "flower") stands for FOR - LET - WHERE - ORDER BY - RETURN. It is the main query expression in XQuery.

Clause	Purpose	Required?
FOR $var in expr	Iterates over a sequence of nodes. $var is bound to each node in turn. Like a loop.	At least one FOR or LET
LET $var := expr	Binds a variable to an expression (entire result, not individual items). No iteration.	Optional
WHERE condition	Filters: only keeps iterations where condition is TRUE. Like SQL WHERE.	Optional
ORDER BY expr	Sorts the results by the given expression. Ascending/descending.	Optional
RETURN expr	Constructs and returns the result for each (filtered) iteration.	Required

XQuery FLWOR Examples:

```
(: XQuery Example 1: Find names of all students from Kathmandu :)
(: (: ... :) = XQuery comments :)

for $s in /university/student
```

```
where $s/address/city = "Kathmandu"
return $s/name/firstname/text()

(: Result: Ram :)
```

```
(: XQuery Example 2: List student id and full name, sorted by lastname :)

for $s in //student
let $fullname := concat($s/name/firstname, " ", $s/name/lastname)
order by $s/name/lastname ascending
return
  <result>
    <id>{ $s/@id }</id>
    <name>{ $fullname }</name>
  </result>

(: Result:
  <result><id>S101</id><name>Ram Sharma</name></result>
  <result><id>S102</id><name>Sita Thapa</name></result>
:)
```

```
(: XQuery Example 3: Count students per city :)

for $city in distinct-values(//student/address/city)
let $students := //student[address/city = $city]
order by $city
return
  <citygroup city="{$city}" count="{count($students)}">
    {
      for $s in $students
      return <student id="{$s/@id}">{$s/name/firstname/text()}</student>
    }
  </citygroup>
```

```
(: XQuery Example 4: Join-like query (students and their courses) :)

for $s in //student,
    $c in $s//course
where $c/@credits > 3
return
  <enrollment>
    <student>{ $s/name/firstname/text() }</student>
    <course>{ $c/text() }</course>
    <credits>{ $c/@credits }</credits>
  </enrollment>
```

XPath vs XQuery vs XSLT Comparison:

Feature	XPath	XQuery	XSLT
Purpose	Navigate and select nodes from XML.	Query and transform XML data.	Transform XML to another format (XML, HTML, text).
Paradigm	Path expressions.	Functional query language (FLWOR).	Template-based transformation rules.
Output	Node sets, values.	XML, text, or any structured output.	XML, HTML, plain text documents.
Turing Complete?	No.	Yes.	Yes.
Relationship	Used inside XQuery and XSLT.	Superset of XPath.	Uses XPath for node selection.
Standard	W3C XPath 3.1	W3C XQuery 3.1	W3C XSLT 3.0
SQL Analogy	Like SELECT / FROM / WHERE fragments.	Like full SQL with aggregation.	Like a stored procedure/report generator.

Q13. What is XSLT (Extensible Stylesheet Language Transformations)? Explain how XSLT transforms an XML document. Write an XSLT stylesheet to convert an XML document to HTML.

[2+4+2]
2081 Ashwin 2078 Chaitra 2074 Bhadra

Answer: XSLT - Extensible Stylesheet Language Transformations

XSLT (Extensible Stylesheet Language Transformations): XSLT is a W3C standard language for TRANSFORMING XML documents into other formats - including HTML for display, plain text for reports, or another XML structure for data conversion. An XSLT stylesheet contains TEMPLATES that match XML elements and specify how they should be transformed.

How XSLT Works:

XSLT Transformation Process

```
   +----------------+      +------------------+      +--------------+
   |   XML Source   |      |  XSLT Stylesheet |      |    Output    |
   |   Document     | +--> |  (Templates +    | ---> |  HTML / XML  |
   | (university.xml)|     |   Transformation |      |  /Text, etc.)|
   +----------------+      |   Rules)         |      +--------------+
                           +------------------+
                                     |
                           +------------------+
```

```
                              |   XSLT Processor   |
                              |  (e.g., Saxon,     |
                              |   libxslt, Xalan)  |
                              +-------------------+

The XSLT processor applies templates from the stylesheet to the XML source
and produces the output document.
```

Note: XSLT is declarative and template-based. Each template matches a node type/pattern and defines the output for matching nodes.

XSLT Stylesheet Example (XML to HTML Table):

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0"
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

  <!-- Template matching the ROOT/document node -->
  <xsl:template match="/">
    <html>
      <head><title>University Students</title></head>
      <body>
        <h2>Student List</h2>
        <table border="1">
          <tr>
            <th>ID</th>
            <th>First Name</th>
            <th>Last Name</th>
            <th>City</th>
          </tr>
          <!-- Apply template to all student elements -->
          <xsl:apply-templates select="university/student"/>
        </table>
      </body>
    </html>
  </xsl:template>

  <!-- Template matching each STUDENT element -->
  <xsl:template match="student">
    <tr>
      <!-- value-of extracts text content -->
      <td><xsl:value-of select="@id"/></td>
      <td><xsl:value-of select="name/firstname"/></td>
      <td><xsl:value-of select="name/lastname"/></td>
      <td><xsl:value-of select="address/city"/></td>
    </tr>
```

```
  </xsl:template>

</xsl:stylesheet>
```

Output HTML produced by applying the above XSLT to the university XML:

```
<html>
  <head><title>University Students</title></head>
  <body>
    <h2>Student List</h2>
    <table border="1">
      <tr><th>ID</th><th>First Name</th><th>Last
Name</th><th>City</th></tr>
      <tr><td>S101</td><td>Ram</td><td>Sharma</td><td>Kathmandu</td></tr>
      <tr><td>S102</td><td>Sita</td><td>Thapa</td><td>Pokhara</td></tr>
    </table>
  </body>
</html>
```

Key XSLT Elements:

XSLT Element	Purpose	Example
<xsl:template match="">	Defines a template rule. match= uses XPath to specify which nodes this template handles.	match="/" for root; match="student" for student elements.
<xsl:value-of select="">	Outputs the TEXT VALUE of the selected XPath expression.	<xsl:value-of select="name/firstname"/>
<xsl:apply-templates select="">	Applies matching templates to the selected nodes (recursion).	<xsl:apply-templates select="//student"/>
<xsl:for-each select="">	Iterates over a set of selected nodes. Like a loop.	<xsl:for-each select="//course">
<xsl:if test="">	Conditional output. Outputs content only if test is true.	<xsl:if test="@credits > 3">
<xsl:choose>/<xsl:when>	Multi-condition branching (like switch/case).	<xsl:when test="city='KTM'">
<xsl:sort select="">	Sorts nodes before processing.	<xsl:sort select="name/lastname"/>

SECTION 6: XML Storage in DBMS and Applications

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2073 Bhadra, 2072 Ashwin

Q14. How can XML data be stored and queried in a Relational DBMS? What are the approaches for XML storage? What are the applications of XML in DBMS?

[2+4+2]
2081 Ashwin 2079 Chaitra 2075 Bhadra

Answer: XML Storage in DBMS and Applications

Approaches for Storing XML in Relational DBMS:

Approach	Description	Advantages	Disadvantages
Store as Text (CLOB)	Store the entire XML document as a character large object (CLOB/TEXT column) in the database.	Simple - no structure needed. Preserves exact XML format.	Cannot query internal structure using SQL. Full document must be loaded to access any element.
Shredding (Relational Decomposition)	Parse XML and DECOMPOSE it into multiple relational tables - one table per element type.	Full SQL querying power. Standard relational operations.	Complex mapping and reconstruction. Schema must be known in advance.
Native XML Storage (XML Columns)	Most modern RDBMS support an XML data type. XML stored natively with built-in XPath/XQuery support.	Best of both worlds - structured storage + XML querying. Index on XML fragments.	More complex than pure relational. Support varies across RDBMS.
XML-Native Database	A database system designed specifically for XML (e.g., BaseX, eXist-db, MarkLogic). XML is the primary data model.	Optimal XML storage and querying.	Not suitable for structured relational data. Limited SQL support.

XML Support in Major RDBMS:

DBMS	XML Support
Oracle	XMLTYPE data type. Full XPath, XQuery, XSLT support. XMLIndex for indexing XML. XMLDB repository.
Microsoft SQL Server	XML data type. FOR XML clause (convert query results to XML). OPENXML (shred XML into rows). XQuery support.
PostgreSQL	XML data type. xpath() function. xmlforest(), xmlelement() for constructing XML. Limited native XQuery.
MySQL	XML functions: ExtractValue(), UpdateXML(), LOAD_XML(). Limited compared to Oracle/SQL Server.

IBM DB2	PUREXML - native XML storage with XQuery and SQL/XML support. Industry-leading XML support.

SQL/XML Standard (ISO/IEC 9075-14):

SQL/XML is an ISO standard that defines how SQL and XML interoperate. Key features:

```
-- SQL Server: Convert query results to XML using FOR XML
SELECT emp_id, ename, salary
FROM Employee
FOR XML PATH(''employee''), ROOT(''employees'');

-- Result:
-- <employees>
--
<employee><emp_id>101</emp_id><ename>Ram</ename><salary>50000</salary></employee
>
-- </employees>

-- PostgreSQL: Query XML data using xpath()
SELECT xpath(''//firstname/text()'', xml_col)
FROM StudentXML;

-- Oracle: Query XML with XMLQUERY
SELECT XMLQUERY(''for $s in /university/student
                 where $s/address/city = "Kathmandu"
                 return $s/name/firstname''
       PASSING xml_data RETURNING CONTENT) AS result
FROM UniversityXML;
```

Applications of XML in DBMS and Systems:

Application	Description	Example
Data Exchange / Integration	XML is the universal format for exchanging data between heterogeneous systems (different DBMS, platforms, languages).	Bank sends account data to government system via XML. Web services use XML (SOAP).
Configuration Files	Applications store configuration in XML for human readability and structured format.	web.xml (Java Servlet config), pom.xml (Maven), applicationContext.xml (Spring).
Web Services (SOAP)	SOAP (Simple Object Access Protocol) uses XML as the message format for web service communication.	REST alternatives use JSON; SOAP still used in enterprise/banking systems.

Document Storage	Content management systems store documents (articles, books, legal documents) as XML.	DocBook XML for technical documentation; DITA for structured content.
RSS / Atom Feeds	News and blog subscription feeds use XML format.	RSS feed: <item><title>...</title><link>...</link></item>
SVG Graphics	Scalable Vector Graphics (SVG) is an XML-based format for 2D vector graphics.	<svg><circle cx="50" cy="50" r="40" fill="red"/></svg>
Office Documents	Microsoft Office Open XML (.docx, .xlsx, .pptx) and OpenDocument are XML-based.	A .docx file is a ZIP archive containing XML files.

Q15. Compare Relational DBMS, Object-Oriented DBMS (OODBMS), and Object-Relational DBMS (ORDBMS). When would you use each?

[2+4+2]
2080 Ashwin 2078
Baishakh 2073 Bhadra

Answer: RDBMS vs OODBMS vs ORDBMS Comparison

Feature	Relational DBMS (RDBMS)	Object-Oriented DBMS (OODBMS)	Object-Relational DBMS (ORDBMS)
Data Model	Tables (relations), rows, columns. Strict 1NF.	Objects with attributes and methods. Classes and inheritance.	Tables extended with complex types, collections, inheritance.
Query Language	SQL (standard, powerful).	OQL (Object Query Language) or host language. Less standardized.	Extended SQL (SQL:1999+) with O-R features.
Type System	Fixed built-in types (INT, VARCHAR, etc.).	Rich user-defined types (classes). Full OOP type system.	User-defined types + built-in types + collection types.
Inheritance	NOT supported natively.	Full class inheritance hierarchy.	Type and table inheritance (UNDER clause).
Complex Objects	Poor support. Must normalize or use workarounds.	Excellent native support.	Good support via structured types, arrays, REF.
Impedance Mismatch	High - OOP apps must convert objects to/from rows.	None - objects stored directly.	Reduced - closer to OOP model.
Performance	Excellent for set-oriented queries.	Good for navigational (pointer-based) queries.	Good for both; optimizer handles complex types.

Maturity/Standards	Highly mature. SQL is standard. All RDBMS vendors.	Less mature. Limited adoption. ObjectStore, Versant.	Mature. SQL:1999 standard. PostgreSQL, Oracle, DB2.
Best For	Business apps, ERP, CRM, financial systems.	CAD/CAM, multimedia, engineering design, scientific apps.	Complex modern apps needing both relational and OOP features.

Most Repeated: "Compare RDBMS, OODBMS, and ORDBMS" or "What is the difference between Object-Oriented and Object-Relational database?" appears frequently. Key points: RDBMS = tables only; OODBMS = objects/methods/inheritance but not relational; ORDBMS = best of both worlds.

QUICK REVISION: Chapter 7 - Key Points Summary

Topic	Key Points	Expected Marks
Complex Types & O-R Model	Traditional RDBMS: only atomic types (1NF). O-R extends with: structured types, collections (ARRAY, MULTISET), type/table inheritance (UNDER), REF types, methods, LOB (BLOB/CLOB).	2-4
Structured Types	CREATE TYPE name AS (attr1 type1, ...) NOT FINAL. Used as column types. Accessed via dot notation. NOT FINAL allows subtypes. FINAL blocks further subtyping.	2-4
Type Inheritance	CREATE TYPE SubType UNDER SuperType AS (...). Inherits all attributes. Models is-a relationships. Polymorphism via substitutability. ONLY() clause for supertable-only queries.	2-4
Table Inheritance	CREATE TABLE SubTable OF SubType UNDER SuperTable. Subtable rows appear in supertable queries. ONLY(SuperTable) excludes subtable rows.	1-3
ARRAY & MULTISET	ARRAY: ordered, indexed by integer, max size declared. MULTISET: unordered bag. Both accessed via UNNEST(). Allow multi-valued attributes (violate 1NF deliberately).	2-4
Object Identity & REF	OID: system-generated unique identity. REF(Type): stores OID pointer to another object row. Navigate with -> (dereference). Scope restricts target table.	2-4
Impedance Mismatch	Gap between OOP objects and relational tables. ORM bridges it: Class->Table, Object->Row, Attribute->Column, Reference->FK. Popular ORM: Hibernate(Java), SQLAlchemy(Python), EF Core(.NET).	2-4

XML Structure	Declaration + Root element + Child elements + Attributes + Text content. Well-formed: proper closing, nesting, quoting, case-sensitive, one root. NOT HTML (display vs data).	3-6
DTD vs XML Schema	DTD: own syntax, only #PCDATA, limited cardinality. XSD: XML syntax, rich types (xs:integer, xs:date), minOccurs/maxOccurs, namespaces, type inheritance. XSD preferred.	3-6
XPath	Path language to navigate XML tree. /root/child: absolute. //element: any depth. @attr: attribute. [predicate]: filter. Axes: child, parent, ancestor, descendant. Functions: count(), text(), contains().	3-6
XQuery FLWOR	FOR (iterate) - LET (bind variable) - WHERE (filter) - ORDER BY (sort) - RETURN (construct output). SQL equivalent for XML. XQuery is Turing-complete. Superset of XPath.	4-8
XSLT	Template-based XML transformation. match= uses XPath. value-of: extract text. apply-templates: recurse. for-each: loop. Output: HTML, XML, text. Used for XML to HTML conversion.	2-4
XML in DBMS	Storage approaches: CLOB (text), shredding (decompose to tables), XML column (native). SQL/XML: FOR XML (SQL->XML), XQuery in SQL. Applications: data exchange, config files, SOAP web services.	2-4
RDBMS vs OODBMS vs ORDBMS	RDBMS: tables+SQL, mature, best for business. OODBMS: objects+OQL, no impedance mismatch, CAD/multimedia. ORDBMS: best of both, extended SQL:1999, PostgreSQL/Oracle support.	2-4

Best of Luck! | TU IOE BCT | CT 652 DBMS | Chapter 7: Object-Based Databases and XML